EnergyWatch AI

GOUVERNANCE

Agent Card

System card and one card per agent: role, tools, permissions, autonomy, human oversight, failure modes and limits.

Projet	**EnergyWatch AI, plateforme multi-agents (LangGraph)**
Secteur	**Énergie**
Cadre	**Mini-projet S5 · Agent en production**
Version	**1.2.0**
Date	**30 octobre 2026**
Dépôt	**github.com/bouincha/Energy-Watch-AI**

Agent Card: EnergyWatch AI

> **Source of truth:** `app/governance/agent_cards.json`, served live at `GET /api/agent-card` and shown on the dashboard's **Governance** page. **Kept up to date automatically:** the test `tests/test_governance_eval.py` fails in CI if an agent in the graph has no card, a card has no matching agent, a required field is missing, a card references an unknown tool, or the card version differs from the application version.

System card

Field	Value
Name	EnergyWatch AI
Version	1.2.0 (alpha, production pilot)
Last updated	2026-10-30
Sector	Energy
Architecture	LangGraph supervisor pattern: guardrails → supervisor → specialists → report_writer → monitor
LLM providers	Groq, Gemini, OpenAI, Anthropic, deterministic mock
Prompt version	`2026.10.1` (`app/agents/prompts.py`)
SLOs	End-to-end latency ≤ 8,000 ms · error/degraded rate ≤ 20 % · availability 99 %
Owner	EnergyWatch AI team (contact: GitHub issues)

Intended use

- Answer energy managers' questions about consumption, cost, CO_2 and peaks
- Detect and explain abnormal consumption (leaks, schedules, drifts, faults)
- Recommend energy-efficiency actions grounded in a curated knowledge base
- Queue proposed actions for human approval with a full audit trail

Out of scope

- Direct control of equipment (the system is read-only on the infrastructure)
- Safety-critical decisions in clinical areas without the facility manager
- Financial or contractual advice on electricity contracts
- Questions unrelated to energy (blocked by the guardrails)

Risks and mitigations (OWASP Top 10 for LLM applications)

Risk	Mitigation
LLM01 Prompt injection	Pattern-based input guardrail before any LLM call; incident RB-003 on bursts
LLM02 Sensitive information disclosure	Output scanning for credentials; PII redaction in logs; no secrets in prompts
LLM06 Excessive agency	Read-only tools; medium/high-risk actions require human approval; audit trail
LLM09 Misinformation / hallucinated numbers	Numbers come from deterministic tools; prompts forbid invented facts; citations
LLM10 Unbounded consumption	Input length limit, per-client rate limiting, timeouts, circuit breaker
Provider outage	Retries + circuit breaker + deterministic degraded mode (RB-001)

Agent cards

1. Guardrails Agent (`guardrails`) · v1.2.0 · deterministic

- **Role:** security gate.
- **Does:** checks every question for prompt-injection patterns, length (≤ 1,000 characters), hidden control characters, and topic (energy only).
- **Tools:** none · **Permissions:** none (pure function).
- **Human oversight:** blocked queries appear in the run explorer (`status=blocked`) and in the audit log.
- **Failure modes:** false positives on unusual wording; new injections that don't match a pattern.
- **Limitations:** regex-based, so it doesn't catch semantically disguised injections.
- **Metrics:** `energywatch_guardrail_blocks_total{category}`.

2. Supervisor / Planner Agent (`supervisor`) · v1.2.0 · LLM

- **Role:** routing and planning.
- **Does:** classifies the intent (consumption, anomaly, advice, report), detects the site and time window, and plans the specialists, using **structured output** validated by Pydantic (`RoutingDecision`).
- **Output sanitization:** only allowed agents, deduplicated and ordered; valid site IDs only; 1-30 days.
- **Fallback:** deterministic keyword planner (also used in mock mode).
- **Human oversight:** the plan and reasoning are shown with every answer.
- **Limitations:** one site per question.

3. Consumption Analyst Agent (`consumption_analyst`) · v1.2.0 · LLM

- **Role:** data analysis with **tool calling**.
- **Tools:** `get_consumption_summary`, `compare_periods`, `get_top_consumers`, `get_data_center_pue` (read-only, validated arguments).

- **Fallback:** rule-based tool selection and a template narrative. Tool errors trigger RB-004.
- **Limitations:** hourly granularity; 30 days of history.

4. Anomaly Detector Agent (`anomaly_detector`) · v1.2.0 · LLM for the explanation only

- **Role:** detection and explanation.
- **Tools:** `detect_anomalies` (robust z-score vs. a per-hour, per-day-type baseline).
- **Output:** events with pattern, duration, excess kWh, cost and severity, plus likely root causes.
- **Quality:** precision 1.00 / recall 1.00 on injected ground truth (see EVALUATION.md).
- **Limitations:** the baseline must be re-learned after structural changes on a site.

5. Energy Advisor Agent (`energy_advisor`) · v1.2.0 · LLM + RAG

- **Role:** grounded recommendations and action proposals.
- **Tools:** `knowledge_base.search` (BM25), `detect_anomalies`, `actions.propose`.
- **Permissions:** may **propose** actions, never execute them.
- **Autonomy policy:** low risk is auto-approved; medium and high risk wait for a human.
- **Human oversight:** Approve or Reject on the Actions page or directly in the chat. Every decision is audited.
- **Limitations:** 7 curated documents; savings are estimates.

6. Report Writer Agent (`report_writer`) · v1.2.0 · LLM

- **Role:** final answer synthesis.
- **Does:** combines the specialists' outputs into Markdown and keeps every number and citation. It flags degraded mode and applies the output guardrail (credential redaction).
- **Fallback:** template report built from the specialists' sections.

7. Monitoring Agent (`monitor`) · v1.2.0 · deterministic

- **Role:** observability, SLOs and incident detection.
- **Does:** wraps every agent to record spans linked to the correlation ID, evaluates each run against the SLOs, updates the Prometheus metrics, and opens or resolves incidents (RB-001 to RB-004).
- **Permissions:** writes to the telemetry stores only.
- **Limitations:** in-memory stores (single instance). JSON logs are the durable record.

Versioning policy

- The **system version** (`app/__init__.py`), the **card versions**, and the **prompt version** (`app/agents/prompts.py`) are bumped together and recorded in `CHANGELOG.md`.
- Any change to a prompt, tool list or autonomy policy requires an update to the card, and the CI test enforces this.